# unfuse: Splitting, Verifying, and Quantizing Multimodal Language Models

Sartaj Bhuvaji

October 2026

**Abstract**

A multimodal language model is an encoder bolted onto a language model: a vision tower that turns pixels into vectors, feeding a decoder that was, underneath, already a text model. `unfuse` takes that apart — keep the language half, keep the encoder half, or keep both separately — as a tensor filter and a rename, with no retraining and no quality loss claimed without a measurement to back it. This report documents three case studies built with it (GLM-4.6V-Flash, Qwen3.8-27B, and a requantized Gemma 4 31B), the verification methodology that makes "no quality loss" a checked claim rather than an assumption, and what each investigation got wrong before it got right. The headline result is incidental to the splitting work: quantizing a model for serving measurably degrades its speculative-decoding acceptance rate, independent of wall-clock, monotonic across three points. This is a technical report, not a claim of novel method — the value is in what was measured and what broke along the way.

## 1 Introduction

A model that reads images is two things glued together: an encoder that turns pixels into vectors, and an ordinary language model that consumes them. The two halves are trained together, shipped as one checkpoint, and in many deployments only one of them is ever used. A text-only fine-tuning run still pays to load a vision tower it never calls. A `llama.cpp` server that only wants the language half still carries the encoder's weights in every GGUF quant. `unfuse` takes the two apart: keep the language model, keep the encoder, or keep both separately. It is a tensor filter and a rename, with no retraining and no distillation.

The split itself turns out to be the easy part. Most multimodal architectures separate cleanly along existing module boundaries — the hard part, and the actual subject of this report, is *proving* the result is unchanged rather than assuming it from the architecture diagram. Section 2 describes the two verification bars this project settled on, and why a single tolerance threshold is the wrong tool for either half of the job. Sections 3–5 are three case studies, in the order they were done, each building on a lesson the previous one left behind: GLM-4.6V-Flash (Section 3) establishes that the split can be bit-exact, not just close, and that most of a widely-quoted memory win was never about the weights at all. Qwen3.8-27B (Section 4) is a much larger, much more popular model whose vision tower is a rounding error — 1.658% of the checkpoint — so the motivation for splitting it has nothing to do with size. Gemma 4 31B (Section 5) is the model that broke the premise of this whole project: its encoder is a single matmul, not a tower, so there is almost nothing to remove, and the investigation pivots from splitting to quantization. That pivot produced this report's one genuinely novel measurement, not a splitting result at all: quantizing a model for serving measurably

costs speculative-decoding throughput, independent of wall-clock, scaling monotonically with how aggressively the target is quantized.

Section 6 collects the lessons that generalize past any one model — config validators, serving-engine quirks, and infrastructure traps that cost real GPU time to find. None of it is dressed up as more than it is. This is a technical report of what was built, measured, and broken, not a claim that any of it is a novel method.

## 2 Methodology: Two Verification Bars

Two different operations run through `unfuse`, and they cannot share a correctness bar.

**A split is pure plumbing.** Dropping a tensor, renaming a prefix, and reconstructing a config do no arithmetic on the weights that survive. The result either reproduces the original model's output bit-for-bit, or the extraction is broken — there is no third option, and no tolerance to tune. `unfuse verify` holds that line: it compares logits between the original and the extracted model on a fixed prompt set and requires `max|d| = 0.000e+00`. A unit test pins the bar there so it cannot drift.

**Quantization is the opposite case.** It rounds every weight it touches, so a non-zero difference from the original is not a bug, it is the entire point of the operation. Scoring it against the same bit-exact bar is meaningless by construction. The tempting fix — pick an absolute tolerance and call anything under it a pass — does not survive contact with a real pipeline: nobody can independently justify the number, and the moment a threshold exists, it will eventually be loosened until whatever checkpoint is on the bench that day passes it. `unfuse agreement` uses a comparative bar instead. Whoever published the quantized build being modified already decided how much divergence from the original is acceptable — they shipped benchmark numbers next to it. Score their checkpoint against the original, score this project's checkpoint the same way, and require the latter to be no worse:

```
unfuse agreement google/gemma-4-31B-it ~/models/mine \
    --baseline nvidia/Gemma-4-31B-IT-NVFP4
```

The two modules deliberately share no code. Merging them would break both: the exact bar would reject every legitimate quantization (all of them move some bits), and the tolerant bar would wave through a broken extraction that happened to land inside whatever threshold survived the merge.

A strict *no-worse-than-baseline* gate is the right test only when the change under evaluation is not supposed to cost anything. It is the wrong test the moment that assumption breaks — Section 5 adds FP8 attention on top of NVIDIA's own NVFP4 baseline, so it cannot pass a strict gate by construction. For exactly that case `Agreement.marginal_cost` reports the *increment* over the baseline instead of a pass/fail verdict, which is the only honest thing to report when a change is known in advance to add some cost in exchange for something else.

**The bar only works if the yardstick is not broken, and checking that costs real effort.** The first scoring run against the Gemma 4 31B quantization (full account in Section 5) produced numbers saying this project's checkpoint *beat* NVIDIA's own build. It was noise: the BF16 reference itself was emitting `France is France is France is…` because `gemma-4-31B-it` is instruction-tuned and the scoring script was feeding it raw completion prompts instead of the chat template. Scoring in that degenerate regime inflates divergence by roughly an order of magnitude in arbitrary directions, which is exactly how a *more* quantized checkpoint came out looking closer to the reference. `agreement` grew a `--chat` mode and a fixed prompt set as a direct result;

`verify` kept its raw prompts, because an extraction has to be bit-exact on any input, degenerate or not — that is a different claim from scoring quantization damage, so the two tools diverge again at exactly the place the exact/comparative split already drew. The general lesson generalizes past this one incident: a result that happens to favor the work already done is the one to distrust most, and it is worth the extra step to find out why before reporting it.

## 3 Case Study: GLM-4.6V-Flash

`zai-org/GLM-4.6V-Flash` (`model_type: glm4v`, `Glm4vForConditionalGeneration`) is the model this project's verification methodology was built against, and the one result published without qualification.

### 3.1 What Is Actually There

The checkpoint's own `model.safetensors.index.json` reports a `total_size` that, read as bytes, implies roughly 5.1B parameters. It is not bytes — for this repo that field reports the parameter count directly. The real figure, confirmed against the sum of every tensor shape and against the shard files' actual byte sizes, is 10,292,777,472 parameters.

| Group | Params | Share |
|---|---|---|
| `visual.blocks.0--23` (ViT, 1536d, 12 heads) | 679,550,976 | 6.602% |
| `visual.merger` | 185,081,856 | 1.798% |
| `visual.downsample` | 25,169,920 | 0.245% |
| `visual.patch_embed.proj` | 1,807,872 | 0.018% |
| `visual.embeddings.position_embedding` | 884,736 | 0.009% |
| `visual.{post_conv,post}_layernorm` | 3,072 | 0.000% |
| **DELETE — `model.visual.*`, 181 tensors** | **892,498,432** | **8.671%** |
| `language_model.layers.0--39` | 8,158,760,960 | 79.267% |
| `language_model.embed_tokens` | 620,756,992 | 6.031% |
| `lm_head` (untied) | 620,756,992 | 6.031% |
| `language_model.norm` | 4,096 | 0.000% |
| **KEEP — 523 tensors** | **9,400,279,040** | **91.329%** |

Table 1: GLM-4.6V-Flash parameter breakdown, verified against tensor shapes.

The surgery is five steps:

1. drop every key under `model.visual.` (181 tensors);
2. rename the `model.language_model.` prefix to `model.`;
3. keep `lm_head.weight` (embeddings are untied);
4. rebuild the config via `Glm4Config`, dropping `mrope_section` and `vision_config`; and
5. set `architectures` to `["Glm4ForCausalLM"]` explicitly — `Glm4Config` will not populate it, and without it `AutoModelForCausalLM` has nothing to dispatch to.

The vocabulary needs no changes: all 151,552 tokens, including the image/video special tokens, carry over whether or not anything multimodal is ever sent to the model again.

### 3.2 Verification: The RoPE Question

The one real risk in this split was positional encoding. GLM-4.6V-Flash's text config carries `mrope_section: [8, 12, 12]`, multimodal RoPE that splits the rotary budget across temporal, height, and width axes — $8+12+12=32$, which is head_dim $128 \times$ partial_rotary_factor $0.5 \div 2$. For text-only input, all three axes carry an identical position index, so mRoPE should mathematically collapse to standard RoPE. That is an argument from the config, not a measurement, and this project does not ship arguments from the config without checking them.

It checks out exactly. Comparing logits between the original multimodal model and the extracted text-only model:

```
max|d| = 0.000e+00 across six prompts, argmax_match=True on all
max|d| = 0.000e+00 at 1,207 tokens
```

Zero divergence, not near-zero divergence, at over a thousand tokens of generation. This is the result the rest of this project's verification discipline is built on: an extraction with the right discipline behind it does not approximate the original model, it reproduces it.

### 3.3 The Inverse Operation

Keeping only the vision tower is the mirror image, and mechanically much simpler. The class `Glm4vVisionModel` already exists in `transformers`, and its submodules map exactly onto what sits under `model.visual.`, so stripping the prefix is the whole surgery — no architecture retarget, no RoPE question. Two facts make it runnable on a laptop rather than a rented GPU: all 181 vision tensors live in shard 4 only (confirmed via the index's `weight_map`), so the download is 4.66 GB rather than the full 20.59 GB, and the result is 1.78 GB. Run on an RTX 2060 (6.44 GB, SM75, so float32, no bf16 support) with 7 GiB of system RAM:

```
181 tensors, 892,498,432 params, 1.785 GB
byte-compared 181 tensors against the source shard -> all bit-identical
max|d| = 0.000e+00 (64, 4096)
448x448 image -> 1024 patches -> 256 soft tokens x 4096d
```

Verification here is deliberately weaker than the removal direction: the full multimodal model does not fit on a laptop, so this compares a directly-loaded reference against the `from_pretrained` checkpoint rather than against the full original. It guards the plumbing — config reconstruction, key naming, the round trip — not the arithmetic, which is identical by construction given the byte-level tensor comparison above. One WSL-specific note: the run reported 7.32 GB peak VRAM on a 6.44 GB card. WSL2's WDDM driver lets CUDA spill into shared system memory; a deliberate 7 GB allocation succeeding confirmed this rather than leaving it as a guess. The same code would OOM on native Linux with the same card.

The output of this direction is a feature extractor, not a model — no language head, no classifier, just embeddings aligned to GLM-4's input space. This asymmetry is worth naming explicitly: removing vision leaves a complete, working model, because a decoder with its own `lm_head` stands alone. Removing text leaves a component that needs something else to consume its output.

### 3.4 GGUF: mmproj Towers Do Not Take K-Quants

Built with `llama.cpp` at master (`GLM4` architecture support). All five quantizations load and generate coherently; no perplexity comparison against the bf16 original was run for this direction.

| Quant | Size (GB) |
|---|---|
| F16 | 18.81 |
| Q8_0 | 10.00 |
| Q6_K | 8.27 |
| Q5_K_M | 7.05 |
| Q4_K_M | 6.17 |

Table 2: Text-only GGUF quantization ladder, GLM-4.6-Flash-text.

The vision half does not take the same ladder. `convert_hf_to_gguf.py --mmproj` only accepts `f32`, `bf16`, `f16`, and `q8_0` — `llama-quantize` and the K-quant ladder apply only to the text GGUF. This is confirmed, not assumed, against what publishers actually ship: unsloth's Gemma 3 and Gemma 4 mmproj builds are BF16/F16/F32 only, and ggml-org's Qwen2.5-VL mmproj ships Q8_0/F16 only. Q8_0 at 1.03 GB is the floor for this tower; there is no Q4_K_M projector, for any model. Building it also needed a staging workaround: `--mmproj` requires the full multimodal architecture and cannot read a stripped `Glm4vVisionModel` checkpoint directly, so the build stages a directory with the original `config.json` plus an index whose `weight_map` references only shard 4 — no second download. `llama.cpp` has no separate `GLM4V` architecture (only `CHATGLM`, `GLM4`, `GLM4_MOE`, with `glm4v` existing only as a vision-projector type), so both the text-only extraction and GLM-4.6V-Flash's own text tower convert to the same `glm4` arch, and the mmproj should pair with either. This is verified from GGUF metadata, not by running end-to-end multimodal inference, which needed more RAM than the build machine had.

### 3.5 Most of the Memory Win Is the Profiler, Not the Weights

The published model card quoted a memory win for the text-only split without ever benchmarking the parent it was derived from — only the derivative had been measured. Closing that gap, on one A100-SXM4-40GB under vLLM 0.27.1 at `--max-model-len 32768`, the split serves 479,280 KV tokens against the parent's 310,512: +54%. The vision tower is only 1.79 GB of 20.59, so that gap cannot be weights. vLLM's own memory profile says where it actually goes:

| | Parent | Parent, `video: 0` | Split |
|---|---|---|---|
| Weights + non-torch | 19.67 GiB | 19.67 GiB | 17.85 GiB |
| Peak activation | 4.82 GiB | 0.97 GiB | 0.21 GiB |
| KV cache | 11.85 GiB | 15.70 GiB | 18.28 GiB |
| KV tokens | 310,512 | 411,552 | 479,280 |
| Engine init | 116 s | 23 s | 11 s |

Table 3: vLLM memory profile, GLM-4.6V-Flash parent vs. text-only split.

The parent's 4.82 GiB peak activation figure is vLLM profiling the multimodal input path against one maximum-size *video* input — not an image — and sizing a 66,242-token encoder cache against it. That memory is reserved for the life of the server whether or not a multimodal request is ever sent, and it costs 105 s of the parent's startup time as well. Which means 60% of the headline gain (101,040

of 168,768 tokens) is available on the *parent*, unmodified, by passing `--limit-mm-per-prompt '{"video": 0}'` at launch, and no surgery at all. The split is worth +16% over a parent configured that way, plus 1.73 GiB of weights that no launch flag reaches. A card claiming the full +54% against a default-configured parent is not technically wrong, but it is comparing against a misconfiguration rather than the best achievable baseline, and both numbers belong in an honest card. (Do not pass `{"video": 0, "image": 0}` together — vLLM 0.27.1 then skips loading the tower entirely and dies in profiling with an `AttributeError`.)

Both checkpoints store KV at an identical 40,960 bytes/token (40 layers, 2 KV heads, head_dim 128, bf16), which is the arithmetic check behind every token figure above — tokens must equal GiB × 26,214.4 — and confirms none of the measured gain is a cheaper cache; it is entirely reserved-memory accounting.

### 3.6 Throughput Did Not Move, and That Is the Result

Eight benchmark rows (two checkpoints, two engines, concurrency 1 and 16): every concurrency-1 row lands between 61.8 and 63.7 tok/s, every concurrency-16 row between 962.7 and 994.9 — a 3% spread across the entire table. Both checkpoints read the same 18.80 GB per decoded token, because decode never touches the resident vision tower regardless of whether it is present, so both sit against the same bandwidth ceiling on the A100's 1.555 TB/s and both land around 77% of it. The trap to avoid when computing that ceiling is charging the parent for its whole 20.59 GB checkpoint rather than the bytes decode actually reads — that lowers its ceiling by 9% and manufactures a throughput advantage for the split that the measurements do not actually show.

Two comparability checks were necessary before any of the above could be reported honestly. vLLM gave both models `max_num_seqs=256` (clamped from `max_num_batched_tokens=2048`), but only prints the clamp at `DEBUG` log level, so confirming the two runs were actually comparable took a second start with identical flags — two unlogged values are unverified, not equal, until checked. SGLang's own derived values were 3,085 for the parent and 4,096 for the split; unequal, but both roughly 200× the tested concurrency, so nothing was throttled in either run. The Qwen 3.8-27B benchmark in Section 4 was not so lucky. And warmup earned its keep on exactly one of the eight rows: SGLang on the split read 50.7 tok/s cold against 63.3 tok/s warm (+24.9%, with TPOT identical at 15.58 ms). Compared against SGLang's warmed parent figure of 63.7, the cold number would have published the split as 20% *slower* than the model it was derived from.

## 4 Case Study: Qwen3.8-27B

`Qwen/Qwen3.8-27B` (`model_type: qwen3_5`, `Qwen3_5ForConditionalGeneration`) is the opposite case from Section 3 in almost every dimension that matters. It is much larger, its vision tower is a rounding error of the checkpoint rather than a meaningful fraction of it, and the reason to split it has nothing to do with size at all: at the time of writing it had 2.95M downloads/month and 12,691 likes on the Hub, roughly 15× the demand of GLM-4.6V-Flash. The split is worth doing because the model is popular, not because it is heavy.

### 4.1 What Is Actually There

The whole checkpoint was measured by reading all 18 shard headers over HTTP range requests — about 200 KB fetched, no weights downloaded, before deciding whether the split was worth

attempting at all.

| Group | Params | Share | GB |
|---|---|---|---|
| `model.language_model.*` (64 layers) | 25,624,600,064 | 92.236% | 51.25 |
| `lm_head` (untied) | 1,271,398,400 | 4.576% | 2.54 |
| `mtp.*` (fused MTP head, 15 tensors) | ∼424,694,784 | ∼1.529% | 0.85 |
| **DELETE — `model.visual.*`, 333 tensors** | **460,730,096** | **1.658%** | **0.92** |
| **Total** | **27,781,427,952** | 100% | **55.56** |

Table 4: Qwen3.8-27B parameter breakdown, measured from shard headers only.

The tower is 1.658% of the checkpoint, against GLM-4.6V-Flash's 8.671%. Removing it saves 0.92 GB out of 55.56 — this is the same lesson Gemma 4 teaches more sharply in Section 5, and the reason to run `unfuse inspect` before assuming any model is worth splitting at all. The actual win here is packaging: the result loads as a plain causal LM, with no vision dependencies and no image-preprocessing code path, not a smaller file.

### 4.2 A False Alarm: `deepstack_visual_indexes`

The config field `vision_config.deepstack_visual_indexes` reads like it should complicate the split — it names tower layers whose features get injected into the language model at several depths, which sounds like vision reaching into the LLM rather than staying cleanly separable. It does not turn out that way. **Zero tensors in the checkpoint carry a deepstack name.** The only trees present are `model.visual.*`, `model.language_model.*`, `lm_head`, and `mtp.*` — all 1,199 tensors accounted for across those four, nothing left unclassified. The indices the config names are resolved entirely inside the visual tree, and the split stays clean. Worth recording precisely because the config *looks* like a problem on first read, and the answer cost exactly one probe to settle rather than an afternoon of architecture study.

### 4.3 A Hybrid, Not a Transformer

`layer_types` is 48 `linear_attention` layers plus 16 `full_attention` layers, with `full_attention_interval: 4` — every fourth layer is full attention, the rest are linear, alongside `linear_conv_kernel_dim: 4`, `linear_num_key_heads: 16`, `linear_num_value_heads: 48`, and `mamba_ssm_dtype: float32`. Vanilla- transformer assumptions about KV cache sizing, GGUF conversion, and engine support do not transfer to this stack, and several of the traps below are exactly that assumption failing in a new place.

### 4.4 The MTP Head Has No Home in `transformers`

`mtp.*` is a fused multi-token-prediction speculative head: 15 tensors, ∼424.7M parameters, `mtp_num_hidden_layers: 1`. `transformers 5.16` defines `Qwen3_5ForCausalLM`, `Qwen3_5TextConfig`, and `Qwen3_5VisionModel`, but no MTP class for this family, so `Qwen3_5ForCausalLM` reports these fifteen tensors as unexpected keys and silently drops them. The split keeps them by default anyway — dropping them removes a real capability, and the MTP-bearing community GGUFs of this model are among its most downloaded builds, so silently

shipping a build without it would be a regression users would not be warned about. `--drop-mtp` exists as an opt-in for anyone who wants the clean load instead.

Loading the split reports 26,895,998,464 parameters against 27,320,697,856 actually in the checkpoint. The difference, 424,699,392, is exactly the MTP head — confirming by subtraction that `transformers` is loading everything else and only silently ignoring this one component. vLLM 0.27.1 does ship `model_executor/models/qwen3_5_mtp.py` and registers `"Qwen3_5MTP"`, so this looked recoverable there. It was not, on either path tested:

| Config `model_type` | Result |
|---|---|
| `qwen3_5_text` (ours) | `NotImplementedError: Unsupported speculative method: 'mtp'` |
| `qwen3_5` (forced) | resolves `Qwen3_5MTP`, then an `AssertionError` on a shape mismatch |

Table 5: Both attempted paths to recovering the MTP head under vLLM.

The cause lives in `vllm/config/speculative.py:516`: the MTP auto-detect fires only for `model_type in ("qwen3_5", "qwen3_5_moe")`, and that code path then reads the *multimodal* config layout, expecting a nested `text_config`. A flattened text-only config makes it fall back to a default `hidden_size` of 4096 against the real value of 5120. So the head costs 0.85 GB and does nothing today — kept anyway, since the alternative is deleting a capability that cannot be recovered without a fresh 55 GB download, and the only cost of keeping it is the 0.85 GB.

### 4.5 The Surgery, and Verifying It Three Times on Two Machines

The surgery:

1. drop every key under `model.visual.` (333 tensors);
2. rename `model.language_model.` to `model.`;
3. keep `lm_head.weight` (untied);
4. build the config from `text_config` via `Qwen3_5TextConfig`, dropping `image_token_id`, `video_token_id`, `vision_start_token_id`, `vision_end_token_id`, and `language_model_only`; and
5. set `architectures` to `["Qwen3_5ForCausalLM"]` explicitly — the same trap `Qwen3_5TextConfig` shares with `Glm4Config` in Section 3.

All six non-default architecture keys (`attn_output_gate`, `full_attention_interval`, `mamba_ssm_dtype`, `mtp_num_hidden_layers`, `mtp_use_dedicated_embeddings`, `output_gate_type`) were verified to survive the round trip through `Qwen3_5TextConfig` rather than assumed to.

The split ran three separate times — once locally, twice on a rented H100, each from a fresh download of the parent — and all three produced an identical 27,320,697,856 params / 54.64 GB result, all three passing the byte-level check on all 866 kept tensors. Reproduction was checked, not assumed, here as everywhere else in this project. Verification for this model is tensor hashes, not logits: `unfuse.verify.compare_logits` is the right tool when the surgery could plausibly change arithmetic, but this one only drops and renames, so a byte equality check over every kept tensor proves every possible input produces an identical output, where matching logits only proves it for the specific prompts tried. Byte hashing is also the only option on a machine that cannot hold a 27B model in RAM at once.

Two implementation details cost real time here. **numpy has no bfloat16**: calling `.numpy()` on a bfloat16 tensor raises `unsupported ScalarType BFloat16` on exactly the dtype the checkpoint

is stored in, which meant the first version of the verifier could not check the one thing it existed to check. The fix is to hash `t.contiguous().view(torch.uint8).numpy().tobytes()` instead — since the claim under test is byte equality, not numeric equality, this also happens to work unmodified for fp8 checkpoints. And **the output sharding should not mirror the input sharding**: reading shard $N$ and immediately writing shard $N$ holds a whole ∼3 GB shard in RAM and dies partway through on a small machine, leaving a checkpoint on disk that still looks plausible but is truncated. Buffering by bytes and flushing on a fixed budget instead means output sharding does not match input sharding — which is fine, because the index file is the authoritative map either way.

### 4.6 Decode Is Bandwidth-Bound, and the Numbers Say So

Worth computing before attributing any single-stream throughput result to engine choice or anything else. An H100 SXM5 has 3.35 TB/s of HBM3 bandwidth, and a dense model at batch 1 reads every weight once per decoded token:

| | Weights | Floor/token | Ceiling | Measured | Of ceiling |
|---|---|---|---|---|---|
| Qwen3.8-27B (parent) | 55.56 GB | 16.59 ms | 60.3 tok/s | 48.8 tok/s | ∼81% |
| Text-only split | 54.64 GB | 16.31 ms | 61.3 tok/s | 47.2 tok/s | ∼77% |

Table 6: Bandwidth ceiling vs. measured single-stream decode throughput.

This is the actual explanation for the flattest result anywhere in this project's benchmark tables: all four concurrency-1 rows, across both engines and both checkpoints, land between 46.6 and 48.8 tok/s, because every one of them moves the same ∼55 GB over the same memory bus regardless of which server is asking for it. Engine choice cannot matter until concurrency amortizes those reads across more than one request at a time, and the remaining 20–25% of headroom at concurrency 1 can only be reached by more bandwidth or fewer bytes (quantization), not a different server or a better flag. This number is approximate by construction — weight traffic only, no KV cache reads counted, and the hybrid stack's 48 linear-attention layers do not access memory the same way the 16 full-attention ones do — and should be read as a band, not a precise figure.

### 4.7 Serving Traps

**vLLM's default `max_num_seqs` will not start this model.**

```
ValueError: max_num_seqs (1024) exceeds available Mamba cache blocks (362).
Each decode sequence requires one Mamba cache block, so CUDA graph capture
cannot proceed.
```

This is the hybrid stack asserting itself: the 48 linear-attention layers each need one Mamba cache block per concurrent sequence, a limit entirely separate from ordinary KV cache sizing. `--max-num-seqs 256` fixes it; any copy-pasted serving command for this model that omits the flag does not work. Separately, the config still carries `mrope_section` and `mrope_interleaved` inherited from the multimodal parent, which vLLM logs as unrecognized rope-parameter keys — harmless so far, but the GLM split in Section 3 explicitly dropped the same field, and this one should too.

### 4.8 What Removing 1.658% Actually Buys

Same H100 80GB, same flags, vLLM 0.27.1:

| | Parent | Text-only | Δ |
|---|---|---|---|
| Model weights | 51.10 GiB | 50.13 GiB | −0.97 GiB (−1.9%) |
| KV cache available | 18.27 GiB | 19.84 GiB | **+1.57 GiB (+8.6%)** |
| KV cache tokens | 277,435 | 301,465 | **+8.7%** |
| Max concurrency @ 32k | 8.47× | 9.20× | **+8.6%** |

Table 7: What a 1.658% weight reduction converts into.

The weight saving itself is small. It converts almost entirely into KV-cache headroom, which is the number actually worth quoting on a model card rather than the raw parameter delta.

### 4.9 The Benchmark Result That Nearly Got Published Wrong

512-in / 256-out, greedy decoding, `ignore_eos`, one shared OpenAI-compatible client, H100 80GB.

| Engine | Model | c=1 tok/s | c=16 tok/s | c=16 TTFT p50 |
|---|---|---|---|---|
| vLLM 0.27.1 | parent | 47.4 | **635.8** | 657 ms |
| vLLM 0.27.1 | text-only | 46.6 | 554.7 | 654 ms |
| SGLang 0.5.18 | parent | 48.8 | 225.3 | 10,585 ms |
| SGLang 0.5.18 | text-only | 47.2 | **628.5** | 715 ms |

Table 8: Cross-engine serving throughput, before and after accounting for warmup and planner artifacts.

Three things to read off this table, in order of how badly the first draft of this result got them wrong.

**Warmup moves the number by 21%.** The first vLLM run on the split measured 457.9 tok/s; the identical server and flags with one discarded warmup pass measured 554.7. Every row above is warmed. An unwarmed row is not comparable to a warmed one, and mixing the two silently flips conclusions about which checkpoint or engine is faster.

**"The text-only split is faster" is false, stated that broadly.** On vLLM the *parent* is 1.15× faster at concurrency 16. On SGLang the split is 2.79× faster — but only because SGLang's planner allocated the parent `max_running_requests=5` against the split's 16 and then starved the parent of concurrency it was never given a chance to use. That is an engine memory-planning artifact, not a property of either checkpoint.

**The real result is variance, not speed.** Across the two engines, the parent's concurrency-16 throughput spans 2.82× (635.8 vs. 225.3); the split's spans only 1.13× (554.7 vs. 628.5) on identical hardware. The split is not faster. It is *predictable*, and predictability — not raw throughput — is what dropping the vision tower actually buys here. At concurrency 1 all four runs land between 46.6 and 48.8 tok/s regardless of engine or checkpoint: the same 64 text layers do the work in every case, single-stream decode cannot tell the models apart, and that flatness is itself the sanity check confirming the surgery did not change the compute path, echoing Section 4.6's bandwidth-bound explanation for exactly this pattern.

## 5 Case Study: Gemma 4 31B — Quantization, Not Splitting

Gemma 4 is the model that broke this project's premise. The 12B variant (`model_type: gemma4_unified`) has no vision *tower* at all — Google replaced the 27-layer SigLIP encoder with a single matmul: 48×48 raw pixel patches through `LayerNorm` → `Linear(6912→3840)` → `LayerNorm`, plus factorized coordinate embeddings, and a 40ms-raw-waveform audio path with no mel spectrogram and no conformer. The whole multimodal path is 52,379,904 parameters: 0.438% of 11,959,730,224, cross-checked against `llama.cpp`'s own `mmproj-F32` build for this model (209.5 MB ÷ 4 bytes = 52.4M, matching). Removing it saves essentially nothing; quantizing the same checkpoint to Q4_K_M takes 23.9 GB down to 7.1 GB, roughly 160× the saving splitting offers. **Only the 12B is encoder-free** — E2B, E4B, 26B-A4B, and 31B are all `model_type: gemma4` with conventional SigLIP towers, so checking `model_type` before applying any recipe from this report to a Gemma 4 variant is not optional.

That realization is what turned this case study from a fourth split into a quantization project. `nvidia/Gemma-4-31B-IT-NVFP4` does have a real, removable 27-layer SigLIP tower (∼576M params, ∼1.15 GB), but reading the checkpoint's own quantization metadata shows it is the smallest of three obvious wins, not the largest:

| Component | State | Params | Size |
|---|---|---|---|
| MLPs (60 layers) | NVFP4, group 16 | 20.8B | 41.6 → ∼11.7 GB |
| Attention (60 layers) | **untouched, BF16** | 7.93B | 15.85 GB |
| Embeddings (tied = `lm_head`) | untouched | 1.41B | 2.82 GB |
| Vision tower (SigLIP, 27 layers) | untouched | ∼576M | ∼1.15 GB |

Table 9: NVIDIA's `Gemma-4-31B-IT-NVFP4`, by component (∼31.5 GB against a real 32.63 GB, consistent with the accounting).

NVFP4 advertises 4-bit weights *and* activations, but the checkpoint's `hf_quant_config.json` excludes 63 modules, 60 of them the attention blocks, listed one layer at a time. Only the MLPs were quantized, leaving unquantized attention as the single largest block in the file. Leaving it BF16 is a defensible choice — NVFP4 quantizes activations too, and attention activations are exactly the outlier-heavy case SmoothQuant and AWQ exist to handle — but it is also a deviation from NVIDIA's own practice elsewhere: their DeepSeek-V4 and Qwen3-Omni builds ship FP4 experts alongside FP8 attention, and `mtq.NVFP4_MLP_WEIGHT_ONLY_CFG` is a named recipe in ModelOpt, so MLP-only is a deliberate configuration rather than an oversight. This project followed three changes from that observation, in order: strip the vision tower (pure deletion, exactly verifiable), take attention weights to FP8 (the real size win), and pair the unpaired MTP drafter (free throughput, and an open question). The outcome: **32.63 GB → 23.00 GB (−29.5%)**, NVIDIA's NVFP4 MLPs copied verbatim, 230 attention projections requantized to FP8, no vision tower, retargeted onto `Gemma4ForCausalLM`, with generation quality preserved. This section is deliberately written as a narrative rather than a result sheet, because most of the value in building it was in the wrong turns.

### 5.1 Decision: Weights-Only FP8, Not W8A8

FP8 has two independent halves with very different risk profiles. The weight half is a pure function of the stored weights — no calibration set, no forward passes, nothing to get wrong beyond scale

arithmetic. The activation half needs calibration data and is exactly where attention's outliers cause trouble. Weights-only still takes attention from 15.85 GB to 7.93 GB and still speeds up batch-1 decode, since decode at batch 1 is memory-bandwidth-bound rather than compute-bound; what it gives up is a prefill win, because BF16 activations cannot feed FP8 tensor cores. This project chose weights-only because it is the version that definitely ships, and because the headline measurement in Section 5.8 wanted exactly one variable moving at a time. A calibrated W8A8 variant remains a sensible follow-up, not a rejected idea.

### 5.2 Decision: Modify NVIDIA's Checkpoint, Not Requantize from BF16

The alternative was starting from `google/gemma-4-31B-it` and running one mixed-precision ModelOpt pass from scratch. Rejected: it needs a 62 GB download and a calibration pass, and the result would no longer be comparable to NVIDIA's own published benchmark numbers, since any accuracy delta could then come from this project's calibration set rather than from the FP8 attention change under test. Working in place on NVIDIA's own checkpoint keeps their calibrated NVFP4 MLPs bit-identical, so exactly one thing changes between the baseline and the result.

### 5.3 What Did Not Work: Loading the Checkpoint at All

`transformers` cannot load this checkpoint. `quant_method: modelopt` is not among the methods it supports (`fp8`, `mxfp4`, `compressed-tensors`, and `quark` are; `modelopt` is not), and it does not error on the unknown method — it ignores the quantization config entirely, loads the packed `uint8` tensors as if they were dense weights, and dies on a shape mismatch:

```
ckpt: torch.Size([21504, 2688]) vs model: torch.Size([21504, 5376])
```

Exactly 2× on the input dimension, which is itself the tell that NVFP4 packs two 4-bit values per byte, and the clue that made the unpacker straightforward to write once noticed.

### 5.4 The Fix That Unblocked Everything: Verify by Expansion

Weight-only quantization is exactly simulatable. An FP4 or FP8 kernel does not multiply in 4 or 8 bits; it dequantizes the stored weight to a wide type first and multiplies against the activations in that wide type. So a checkpoint whose weights have been round-tripped through the format computes bit-identical results to the real deployed kernel, and accuracy can be measured on an expanded BF16 copy without ever running the quantized kernel itself. Two consequences followed directly: **NVFP4 accuracy work does not need Blackwell hardware**, only throughput measurement does, which overturned this project's earlier plan (that plan had assumed scoring required SM100); and the missing `transformers` loader became a non-issue, since expansion sidesteps it entirely. What expansion does *not* simulate is NVFP4's activation quantization, so an expanded NVFP4 checkpoint is very slightly optimistic relative to a true end-to-end deployment. Both sides of every comparison in this section are expanded the same way, so the comparison stays fair for the specific weight change under test, but none of the numbers below are a claim about deployed accuracy on real NVFP4 tensor cores.

NVFP4's on-disk layout, confirmed against the real tensors rather than assumed from the spec:

```
weight [out, in/2] uint8 two e2m1 values per byte, low nibble first
weight_scale [out, in/16] float8_e4m3 one per group of 16
```

```
weight_scale_2 scalar float32 one per tensor
w = e2m1(nibble) * weight_scale * weight_scale_2
```

Two checks run before trusting any downstream number from the dequantizer: `model.layers.{0,30,59}.self_attn.q_proj.weight` must be bit-identical (`max|d| = 0.0000`), since attention was never quantized by NVIDIA and must survive stripping and renaming untouched; and `weight_scale.max()` $\times$ `weight_scale_2` $\times$ 6 must reproduce the original tensor's absmax exactly (measured: 0.2617), the cheapest possible identity check that the largest e2m1 magnitude times the largest block scale reproduces the original range. Run both before trusting anything else this section reports.

### 5.5 Five Bugs the Real Checkpoint Found That Local Tests Did Not

1. **`*self_attn*` matches `q_norm` and `k_norm`.** An early run quantized all 120 of them. They are 1-D RMSNorm scales — no size to win by quantizing them, and among the most precision-sensitive weights in the model. Caught only because the converted tensor count came to 350 where projections should total 230; fixed by targeting `*_proj` specifically and structurally refusing any 1-D tensor, since the pattern had already proven easy to get wrong once.

2. **The quantization schema is a hybrid.** `quantization_config` carries modelopt keys (`quant_algo`, `quant_method`) and compressed-tensors keys (`config_groups`, `ignore`, `kv_cache_scheme`) in one block. Reading only the modelopt half silently loses the group size, which is nested inside `config_groups`, and the KV cache format, which is named differently in each half.

3. **Tied embeddings defeat name matching.** The exclusion list names `lm_head`, but the tensor ships under the key `embed_tokens`, so matching on names alone reports BF16 weights as already quantized. The dtype recorded in the tensor header is the authority; the config is only a claim about it.

4. **Renaming a prefix invalidates the exclusion patterns.** Moving `model.language_model.*` to `model.*` leaves every `ignore` entry pointing at a path that no longer exists after the rename, so they match nothing, and the next reader of the checkpoint concludes attention was quantized when it was not.

5. **Patterns matched full tensor names, not modules.** Any user-supplied pattern without an explicit `.weight` suffix would silently convert nothing at all, passing without any visible error.

`230`, incidentally, is not a typo for 240: Gemma 4 sets `attention_k_eq_v`, so ten layers share K and V and have no separate `v_proj` — worth checking the architecture before assuming a tensor count is simply wrong.

### 5.6 What Did Not Work: Scoring on Raw Prompts

The first scoring run produced clean-looking numbers claiming this project's checkpoint *beat* NVIDIA's own build, and it passed the "no worse than baseline" gate outright. It was entirely noise. The tell was in the generations themselves: the BF16 *reference* model was emitting `France is France is France is…` The control experiment that cleared it: the untouched original produces the identical degenerate output on that exact prompt, and through the chat template both

the original and this project's copy correctly answer `Paris`. So the bug was never in the extraction — `gemma-4-31B-it` is instruction-tuned, and the scoring script was feeding it raw completion prompts rather than the chat template it was trained to expect. Scoring in that degenerate regime inflates divergence by roughly an order of magnitude in arbitrary directions, which is precisely how a *more* quantized checkpoint managed to come out looking closer to the reference than a less-quantized one did. `agreement` grew a `--chat` mode and a fixed `CHAT_PROMPTS` set as a direct result; `verify` kept its raw prompts unchanged, because an extraction must stay bit-exact on any input, degenerate or not, which is a different claim from scoring quantization damage and belongs to the other bar described in Section 2. The general lesson generalizes past this one incident: a plausible result that favors the work already done is the one to distrust most.

### 5.7 Do Not Read KL as Benchmark Accuracy

A KL divergence of 0.145 looks alarming set next to NVIDIA's own published GPQA numbers, 85.35% vs. 85.80%, which *is* genuinely near-lossless. There is no contradiction between the two: KL computed over a 262k-token vocabulary is far more sensitive than a multiple-choice accuracy metric, and a 9.5% relative weight error at 4 bits can leave benchmark scores essentially unchanged while still producing a KL number that looks large in isolation. This nearly triggered a false alarm about a bug in the dequantizer before the comparison was reframed correctly: judge one quantization against another quantization, never against an absolute divergence number with no reference point.

Measured through the chat template against the BF16 reference:

| | Worst top-1 | Worst KL |
|---|---|---|
| B — NVIDIA NVFP4 | 0.7826 | 1.448e-01 |
| C — ours, +FP8 attention | 0.9130 | 1.716e-01 |

Table 10: Divergence from the BF16 reference, NVIDIA's build vs. this project's build.

C adds 1.19× baseline's KL divergence. Its top-1 agreement came out *higher* than B's on all six test prompts, which should not happen for a strictly more quantized model — six prompts and roughly 150 token positions is a small enough sample that this pattern is plausibly the sample size talking, not a real effect. Both rows should be read as indicative, not final, until re-run on a larger prompt set.

### 5.8 The Headline: Quantizing the Target Costs Speculative Acceptance

The open question this investigation was actually built to answer. The drafter model (`google/gemma-4-31B-it-assistant`, 0.47B parameters) was trained against the BF16 target; quantizing the target moves its output distribution away from what the drafter learned to predict. Speculative decoding stays exact by construction regardless — the target verifies every drafted token and discards whatever it would not have produced itself — so this cannot change output quality. It changes throughput, and under exact speculative decoding throughput is entirely a function of acceptance rate. Measured as tokens accepted per target forward pass — the arithmetic ceiling on speedup, and a property of the two distributions rather than of any kernel implementing them:

**Monotonic in the amount of quantization applied**, which is what makes this result credible where the top-1 numbers in Section 5.6 were not. A published "up to 3×" speedup claim for the

| Target | tok/fwd | vs. BF16 |
|---|---|---|
| R — base BF16 | 2.220 | — |
| B — NVIDIA NVFP4 | 1.655 | −25.4% |
| C — ours, +FP8 attention | 1.247 | −43.8% |

Table 11: Speculative-decoding acceptance, tokens per target forward pass, by how much the target is quantized.

BF16 pairing does not transfer to a quantized target: NVIDIA's own build already surrenders a quarter of the benefit before this project's FP8 attention change adds anything on top of it.

Three caveats that matter to how far this generalizes. The wall-clock column should be ignored entirely: the BF16 reference measured a 0.97× "speedup" — speculative decoding running *slower* than plain generation — because `transformers`'s assisted-generation overhead dominates at this model scale, a property of the measurement harness, not of either model, and exactly why tokens per forward pass is the number reported instead. The measurement used `assistant_model` draft-model speculation, not necessarily the native MTP path the drafter ships for. And it covers four prompts at 96 new tokens each — directionally clear, not a precise coefficient to quote past one decimal place. The practical reading stands regardless of those caveats: if throughput is the reason to want speculative decoding at all, quantizing the target is not free, and the trade against 30% smaller weights is a real cost to weigh, not an assumption to make.

### 5.9 The KV Cache, and a 2× Error in the First Estimate

The initial analysis assumed uniform layers and claimed 80.0 KB/token, 21.47 GB at full context — "as large as the weights." That was wrong, and it was wrong because Gemma 4's layers are heterogeneous, a fact that surfaced only because `transformers` raises `AmbiguousGlobalPerLayerAttributeError` on a global read of `num_key_value_heads` rather than silently returning a number for the wrong layer type:

```
10 x full_attention kv_heads=4 head_dim=512 -> 40.0 KB/token
50 x sliding_attention kv_heads=16 head_dim=256 -> capped at ~0.4 GB by the window
```

The counter-intuitive part is that the sliding layers carry four times the KV heads of the global ones, but their 1024-token window caps them at a fixed ∼0.4 GB regardless of how long the context runs — only the ten global layers actually grow with context length: 1.75 GB at 32,768 tokens, 5.79 GB at 131,072, 11.16 GB at the full 262,144-token window. So at the full window the cache is roughly half the quantized weights, not equal to them as the first, uniform-layer estimate claimed. Worth knowing before optimizing further, but it does not outrank the weight savings reported above, and the stronger original claim was purely an artifact of assuming a homogeneous model where none exists.

### 5.10 Two Things Reached the Published Repo Before Being Caught

Both are the same bug wearing different clothes: metadata that describes the model *before* the surgery, carried forward as if it still applied after. **Loose files travel with a checkpoint.** The copy step that carried non-safetensors files forward brought NVIDIA's own `README.md` (describing their build, not this project's result) and a 2,155-line ModelOpt quantizer dump listing the vision tower

and reporting attention as unquantized — both landed in the upload, and neither was true of the actual result shipped. There is now a `STALE_METADATA` skip set, with a test asserting the tokenizer and chat template still travel with the checkpoint, so the guard does not become an excuse to drop files the model genuinely needs to load. **Promoting a nested config exposes fields to new validators.** Gemma 4's `text_config` carries `num_experts: null`, `expert_intermediate_size: null`, and `top_k_experts: null`, all inert because `enable_moe_block` is `False`. Nested inside `text_config` nobody checks them; promoting `text_config` to the top level exposes them to the Hugging Face config parser, which rejects `"num_experts" must be a number`. The fix — dropping the three inert keys — was only trusted once `Gemma4TextConfig` was constructed both ways and every resolved field was asserted to match; dropping config keys because they "look unused" is otherwise a guess, not a verified fix.

### 5.11 vLLM and This Checkpoint: Three Wrong Answers Before the Right One

The published model card told people to run `vllm serve --quantization modelopt`. That instruction was written from what the format's documentation implies, never actually run, and it does not work. Correcting it took two rented GPU instances and about $7, mostly because the first two candidate explanations were both wrong and both *felt* conclusive at the time. The final answer: this checkpoint cannot be served by vLLM at all today, for a reason that has nothing to do with Gemma 4 or with vLLM's Gemma support specifically, and everything to do with the weights-only FP8 decision recorded earlier in this section.

#### *The first wrong answer: "vLLM doesn't support Gemma 4"*

The observed error was `AmbiguousGlobalPerLayerAttributeError: 'head_dim' is a per-layer attribute`, and the conclusion drawn from it was an upstream architecture support gap. That conclusion cost nothing to check and was false: a file listing of the release tag — no GPU, no download, one `curl` call — shows `gemma4.py`, `gemma4_mm.py`, and `gemma4_mtp.py` all present in v0.27.1, with `git log` dating the first of them four months before that release. *Before renting a GPU to explain a failure, check whether the cheapest possible observation already falsifies the explanation.*

#### *The second wrong answer: "it's a transformers version skew"*

Falsifying a cause is not the same as finding one. The next hypothesis came from two pieces of evidence inside the error itself: the exception's home library is `transformers`, not vLLM, which is evidence about which side of the stack changed; and vLLM 0.27.1 pins `transformers>=5.5.3` with no upper bound while unfuse requires `transformers>=5.15.1`, so vLLM was necessarily running against a transformers version newer than anything it had been tested with. That hypothesis is testable on the config alone, with no weights and no GPU needed, and a version sweep bisected it exactly:

```
5.5.3, 5.8.0, 5.11.0, 5.13.1, 5.14.0, 5.14.1 head_dim = 256 global read OK
5.15.0, 5.15.1 AmbiguousGlobalPerLayerAttributeError
```

Pinning `transformers==5.14.1` does clear the config error, and that is a real, necessary finding affecting any Gemma 4 31B on vLLM, Google's own build included — but it was not the actual blocker. It only cleared the way to the next failure, an `AssertionError` deep inside the QKV loader.

*Fixing the first error tells you what the second error is, not that you were right about the cause.* The `transformers` guard turned out to be correct, not a regression: Gemma 4 31B genuinely has two head dimensions, 256 on the 50 sliding layers and 512 on the 10 full-attention layers, so a global `head_dim` read is meaningless by construction, and vLLM 0.27.1 reading it globally returns 512 — wrong for 50 of 60 layers.

### *The control experiment that inverted the conclusion*

The question worth answering was not "why does this fail" but "is this ours?" The control is running the unmodified upstream artifact through the identical serving path:

```
nvidia/Gemma-4-31B-IT-NVFP4 -> SERVED OK
sartajbhuvaji/Gemma-4-31B-IT-NVFP4-FP8attn-text -> AssertionError in load_qkv_weight
```

Upstream serves, on an A100, not even on Blackwell hardware. So vLLM can serve Gemma 4 31B, the transformers pin is sufficient for that, and the blocker is something this project's build introduced, not an upstream gap. *A control run on the unmodified upstream artifact is what separates "their bug" from "ours," and it is worth its cost early rather than late.*

### *Instrumenting instead of guessing*

The assertion carried no shapes in its message, so the loader was monkeypatched to print them at the exact point of failure rather than reasoning about what they might be from the stack trace alone:

```
shard=k param=(16384, 2688) loaded=(4096, 5376)
```

$2688 = 5376/2$ is the tell — the same $2\times$ factor that identifies FP4 packing everywhere else in this section. vLLM had allocated the attention parameter as if it were NVFP4-packed, while the stored tensor is actually full-width FP8: vLLM was applying NVFP4 to attention, meaning it had never read the `quantized_layers` mapping that says otherwise. Reading the dispatch source rather than the documentation found why — two exact-match gates, neither visible from the config file alone: `quant_algo` is literally the string `"NVFP4"`, and vLLM's mixed-precision path is reachable only when that string equals `"MIXED_PRECISION"` exactly, so it was never entered; and `quantized_layers` keys in the checkpoint are globs like `model.layers.0.self_attn*`, but the lookup vLLM performs is exact dict membership with no glob expansion, so they match nothing. The correct form names each projection individually: 230 FP8 entries beside 180 NVFP4 MLP entries. *When a config key appears to be silently ignored, read the dispatch code — the gate is often a literal string comparison, and no amount of plausible-looking JSON will pass it.*

### *The real blocker, and why it is the worst kind*

With both defects above patched, the model loaded, ran, and generated fluent nonsense:

```
>>> What is the capital of France? Answer in one word.
same dePK singleale dePK singleale dePK singleale ...
```

`ModelOptFp8LinearMethod.create_weights` registers `input_scale` as a required per-tensor parameter, and `process_weights_after_loading` calls `layer.input_scale.max()`. vLLM's ModelOpt FP8 path is W8A8 only. This checkpoint is deliberately W8A16 — FP8 weights, BF16 activations, chosen for exactly the accuracy reasons recorded earlier — so there is no `input_scale` tensor to load, and the scale stays at its initialization sentinel. Nothing errors, because FP8 and BF16 weights have identical tensor shapes: every shape check in the loader passes, and only the arithmetic downstream of it is wrong. This is strictly worse than the crash it replaced — a crash self-reports, and a checkpoint that serves confident garbage does not. *Prefer the failure mode that announces itself: a dtype swap that preserves tensor shape will pass every structural check a loader makes, and fail silently past all of them.*

So the decision that is correct on accuracy grounds (Section 2's earlier discussion) is exactly the one with no serving path today. Fixing the two dispatch defects in isolation would convert a loud failure into a silent one, which is worse, not better — they are worth fixing for metadata correctness regardless, but `quant_algo` should stay `NVFP4` until either a calibrated W8A8 variant exists or vLLM gains FP8 weight-only support in the ModelOpt path specifically.

### *What actually runs, tested*

Stock released vLLM, not a nightly build — "install the nightly" is a materially different instruction to publish on a model card than a pinned release:

```
pip install vllm==0.27.1 'transformers==5.14.1'
python tools/dequantize.py ./ckpt ./ckpt-bf16
vllm serve ./ckpt-bf16
```

On an H100 PCIe 80 GB: engine up in 103 s, 36.5 tok/s on a batch of 5, eager mode. Correct answers on every test prompt: `Paris`; `408` with correct working shown; `Das Wetter ist heute kalt.`; Rayleigh scattering explained correctly; `my_string[::-1]`. A vLLM nightly serves the same expansion at 42 s and 48.8 tok/s. Expansion costs back the size saving — 23 GB becomes 61 GB — so it needs an 80 GB card regardless of which vLLM build serves it.

### *Corrections to claims this project had already published*

Three claims in earlier versions of this project's own documentation turned out to be wrong, corrected only once the vLLM investigation above forced a closer look. **"NVFP4 executes only on Blackwell (SM100+)" is false under vLLM**: it selects `MarlinNvFp4LinearKernel`, and `ModelOptMixedPrecisionConfig.get_min_capability()` reports 75, with a comment in the source recording validation on a T4 (SM75) and an A100 (SM80). Blackwell provides native FP4 tensor cores; without them, Marlin repacks to W4A16 at load time, so the *stored* size saving is not automatically a *runtime* memory saving — a 40 GB A100 ran out of memory loading this 23 GB checkpoint for exactly this reason. **The FP8 KV cache needs SM89+, not Blackwell**; below that, vLLM refuses to start with `FP8 KV cache is not supported by the Triton attention backend` and names its own fix, `--kv-cache-dtype bfloat16`. And **vLLM 0.27.1 has no `quantized_layers` support at all** — it parses only `exclude_modules` (the legacy `hf_quant_config.json` field) or `ignore`. NVIDIA's own build serves correctly on this release specifically because its 60 attention blocks sit in `exclude_modules`, which 0.27.1 does read; mixed precision as this project configured it cannot be expressed to that vLLM release in any form.

***The meta-lesson***

Two of the three explanations along the way were wrong, and each one was written down with real confidence before the next observation demolished it, including in this project's own earlier published documentation. The pattern that eventually worked: the cheapest falsifying observation first, then a control run against the unmodified upstream artifact, then instrumentation at the exact point of failure, then the serving library's own dispatch source — and only after all four, a claim, tested against the exact software versions named in it.

## 6 Cross-Cutting Lessons

The three case studies above each have their own narrative; this section pulls out the lessons that generalize past any one of them, grouped by where they actually bite rather than by which model surfaced them first. Each cost real time or real GPU-hours to find.

### 6.1 Config and Metadata Traps

**A config field can describe the model before the surgery, not the model that actually ships.** This is the single most common failure pattern across every case study in this report: `deepstack_visual_indexes` in Section 4 looked like a problem and was not; the exclusion-list name mismatch and the invalidated `ignore` prefixes in Section 5 both looked like correct configuration and were not; the loose `README.md` and quantizer dump that traveled into a published repo in Section 5 described NVIDIA's build, not this project's result. The fix is the same every time: verify against the tensor headers and the dispatch code, not against what the config claims.

**A class that does not populate `architectures` leaves nothing for `AutoModelForCausalLM` to dispatch to.** True of `Glm4Config` (Section 3) and of `Qwen3_5TextConfig` (Section 4), independently, on two unrelated model families. Setting `architectures` explicitly after reconstructing a config is not an edge case to remember occasionally; treat it as a required step of any split.

**`total_size` in `model.safetensors.index.json` is not always bytes.** For GLM-4.6V-Flash it reports the parameter count instead (Section 3). Verify against real file sizes rather than trusting the field name.

### 6.2 Serving-Engine Quirks

**Each engine's own benchmark tool disagrees with every other one's.** `vllm bench serve` and `sglang.bench_serving` sample prompts differently, warm up differently, and define "duration" differently. Run the Qwen3.8-27B comparison in Section 4 against each tool's own harness and SGLang appeared to beat vLLM on time-to-first-token by 3×; run both against one shared OpenAI-compatible client over plain HTTP instead, and vLLM won by 1.7×. Cross-engine numbers are only meaningful from a single shared harness. Pin output length with `ignore_eos` as well, or a chattier model's verbosity becomes the throughput measurement instead of the thing actually under test.

**NVFP4 does not require Blackwell, and FP8 KV cache does not require it either, at a different threshold.** vLLM selects `MarlinNvFp4LinearKernel` and runs it from SM75 up; without Blackwell's native FP4 tensor cores, Marlin repacks to W4A16 at load time, so a checkpoint's *stored* size saving is not automatically a *runtime* memory saving, and a 23 GB NVFP4 checkpoint OOM'd

a 40 GB A100 for exactly this reason (Section 5.11). The FP8 KV cache separately needs SM89+; below that, pass `--kv-cache-dtype bfloat16`.

**Installing a tool into a venv does not put it on `PATH`.** vLLM shells out to `ninja` at build time and fails with a bare `FileNotFoundError` surfaced three stack frames deep as `Engine core initialization failed`. Activating the venv puts `$VENV/bin` on `PATH`; running `$VENV/bin/vllm` directly does not, so a correctly installed `ninja` stays invisible to the engine that needs it. Separately, vLLM and SGLang pin different torch builds, and installing both into one venv lets the second silently downgrade the first — always give them separate environments.

**The reasoning field has two names.** vLLM 0.27.1 returns a model's thinking trace as `message.reasoning`; SGLang returns the same concept as `message.reasoning_content`. Client code written against one silently drops it on the other with no error.

**A daemon thread holding CUDA can abort the process at exit, after every check has already passed.** Streaming generation handed to a `Thread(..., daemon=True)` can still be inside `generate()` when the interpreter tears down CUDA at exit, producing `terminate called without an active exception` and a non-zero exit code after every metric has already been computed and written to disk. Non-daemon, plus an explicit `join()`, avoids it.

### 6.3 Infrastructure and Process Traps

**A background job's own echo is not evidence it started.** `pip install huggingface_hub[cli]` warns that it installed to `~/.local/bin`, which is not on `PATH` on a fresh cloud image. A backgrounded `nohup hf download ... &` then fails with `nohup: failed to run command 'hf'` *inside the log file*, while the launching shell happily reports the download as started. Twenty minutes of a rented GPU instance went to a download that never began. Use absolute paths for anything backgrounded, and verify bytes actually landing on disk rather than trusting a shell's own echo that a command launched.

**`pkill -f <pattern>` can match your own SSH session.** The remote command string lives in the remote shell's own `argv`, so `pkill -f pipeline.sh` issued over SSH can kill the shell that is running the SSH session itself — connection drops, exit 255, and whatever was mid-flight dies with it. This happened three times before being tracked down. The `[p]attern` bracket trick only helps when the pattern string is not also present verbatim somewhere else in the full command line; a heredoc containing the same string defeats it too. Kill by PID instead, or write the command to a file and `scp` it over rather than passing it inline.

**`hf upload` creates the destination repo public by default, with no prompt.** A 54.64 GB checkpoint began publishing to a public repository before its model card existed to explain what it was. Create the destination repo explicitly ahead of time, or check `model_info(...).private` immediately after upload starts and flip it if it is wrong.

**A backgrounded job that must outlive the SSH session needs `setsid`.** Plain `nohup ... &` still dies when its parent session is killed; `setsid nohup ... < /dev/null & disown` survives a dropped connection.

**A `.env` file edited on Windows silently breaks every API key read from it.** Files under `/mnt/c` in WSL carry CRLF line endings; sourcing the file leaves a trailing `\r` on the last value read, and the service reports `invalid-api-key` for a key that is, character for character, correct. `file .env` reports `CRLF line terminators` directly; `sed -i 's/\r$//' .env` fixes it, and the launch tooling now strips CRLF on every read rather than trusting the file to already be clean.

**`--system-site-packages` mixes apt and pip packages badly.** A venv built with that flag inherited apt's torchvision 0.22 while pip separately installed torch 2.13, so `torchvision::nms` failed to register — surfacing as a completely unrelated-looking `ModuleNotFoundError: Could not import module 'Gemma4Config'`. pip then skipped installing its own torchvision entirely, since apt's copy already "satisfied" the version requirement, and apt's scipy then collided with the venv's numpy on top of that. The fix was a clean venv with no system packages and torch plus torchvision installed from the same CUDA wheel index together.

**A shell script without a shebang runs under `/bin/sh`, not bash.** A setup script used `exec > >(tee ...)`, a bash-only process substitution; under `sh` it dies instantly and leaves no log at all, because the redirect that would have recorded the error was itself the thing that failed.

**Lambda's `POST /ssh-keys` with a null `public_key` does not error.** It silently generates its own keypair instead and returns the private half in the API response body, which nobody necessarily saves. `ssh_key_names` is fixed at instance launch time, so discovering this after launch means terminate and relaunch, not a fix in place. The root cause was a shell bug, not an API quirk: `python3 -c "..." P="$V"` passes `P=` as a script *argument*, not an environment variable, so the key content silently never reached the script that was supposed to read it. Environment assignments must precede the command they apply to. This project's launch script now verifies the registered key against the local `.pub` file before ever launching an instance against it, rather than trusting that registration worked.

## 7 Conclusion and Future Work

Three models, three different reasons to split or not split them. GLM-4.6V-Flash (Section 3) is the clean case: a genuinely separable vision tower, a bit-exact split, and a memory-win claim that needed correcting once the parent was actually benchmarked rather than only the derivative. Qwen3.8-27B (Section 4) is popular enough that packaging, not size, is the whole argument for splitting it at all — its tower is 1.658% of the checkpoint. Gemma 4 31B (Section 5) is the model that rejected the premise outright: its "tower" is a single matmul, splitting saves almost nothing, and the actual investigation became quantization, with the speculative-decoding acceptance result in Section 5.8 as the one measurement in this report that was not really about splitting at all. The throughline across all three is not a method; it is a discipline — `unfuse verify`'s bit-exact bar for operations that should not change arithmetic, `unfuse agreement`'s comparative bar for operations that necessarily do, and a running habit of distrusting any result that happens to favor the work already done, demonstrated concretely in Sections 2 and 5.6, independently, on two different models.

### 7.1 Why Not to Prune for Size Instead

Worth stating directly, since a reasonable alternative to splitting is pruning the language model itself. The literature argues against it here. At 15% backbone compression, structural pruning of vision-language models degrades multimodal performance by 9.13% while text-only performance drops only 1.07% [Huang et al., 2026]: vision-language alignment is the first thing to degrade, and it degrades without being specifically targeted, so hunting for vision-specific neurons to prune surgically is the wrong level of precision — width-pruning generically and letting vision-serving capacity die along with it is both simpler and consistent with what actually happens. Muralidharan et al. [2024] remains the credible path to a genuinely smaller model instead of a repackaged one, having demonstrated exactly a 12B→8B reduction (Mistral NeMo to MN-Minitron-8B, MMLU 69.5

against the teacher's own 68.0), but width-pruning only, never depth — layer-dropping destroys GSM8K and HumanEval while leaving MMLU looking deceptively unaffected. A ≤30% target is the credible range, with 45% the measured tipping point, at a real cost of roughly 18k H100-hours at the 94B-token retraining tier (∼$40k), or $150–300k for a full from-scratch replication — not a project-weekend undertaking. And multimodal adaptation specifically suppresses text ability in ways that do not reverse cheaply: Irawan et al. [2026] documents a loss on text-only tasks that is "hard to undo with further fine-tuning," with existing remedies needing added adapters or alignment modules rather than a lightweight pass. There is no dormant text-only model sitting underneath a multimodal one's weights, waiting to be fine-tuned back out. Splitting a model that already has a separable encoder gets the model this project actually wants without any of that cost; pruning one that does not is a different, much more expensive project.

### 7.2 What Is Still Open

**The MINER ablation nobody has run on an encoder-free model.** Huang et al. [2024] found that roughly 2% of FFN neurons become modality-specific in encoder-*based* multimodal models, concentrated in shallow layers, and that deactivating that 2% collapses multimodal performance (Qwen2-VL: 0.56 → 0.24). It never ran the converse direction — ablate the vision-specific neurons, then measure *text-only* performance, to see whether any serving capacity for vision was quietly living in the shared FFN all along — and nobody has run either direction on an encoder-free model. Every published modality-pruning result in this space studies LLaVA, InternVL, or Bunny, all of which have separable vision towers; Gemma 4 12B (Section 5) does not, by construction. The script for this exists, scores FFN neurons by activation ratio on image vs. text tokens, zeroes the top slice, and watches both NLLs move. If vision NLL rises while text NLL holds steady, vision-serving capacity emerged in the shared FFN despite the encoder-free architecture, which would mean this report's 0.438% ceiling for Gemma 4 12B's removable multimodal footprint is the wrong number to quote. It has never been run, and is kept outside this project's repository until it produces a result worth reporting, following the same discipline applied to every other number in this report. Estimated cost: roughly one GPU-day.

**The FP8-attention serving path.** Section 5.11 leaves this project's best Gemma 4 31B checkpoint correct on accuracy grounds and unservable by any released vLLM build, because the W8A16 choice that is right for accuracy has no supporting kernel path in vLLM's current ModelOpt integration. Two ways forward exist, and neither has been attempted yet: calibrate a true W8A8 variant, accepting the activation-quantization risk this report deliberately avoided, or wait for upstream FP8 weight-only support in vLLM's ModelOpt path. Until one of them lands, the published checkpoint runs only through `tools/dequantize.py` back to BF16, at the memory cost that implies.

**The MTP pairing, verified only from metadata.** Section 4's unpaired MTP drafter has been verified to pair correctly from GGUF metadata alone; it has not been end-to-end tested serving actual multimodal-free inference, because the build machine available had 7 GiB of RAM and could not hold both the text GGUF and the mmproj simultaneously. That remains an open verification, not a known result.

None of the above is advanced as more than it is. This report is a record of three models taken apart, the two bars that made "unchanged" a checked claim rather than an assumed one, and one measurement that would not have been found by looking for it directly — it fell out of asking why a model that should obviously run faster quantized sometimes does not.

## References


Kaichen Huang, Jiahao Huo, Yibo Yan, Kun Wang, Yutao Yue, and Xuming Hu. Miner: Mining the underlying pattern of modality-specific neurons in multimodal large language models. *arXiv preprint arXiv:2410.04819*, 2024.

Yiran Huang, Lukas Thede, Massimiliano Mancini, Wenjia Xu, and Zeynep Akata. Structural pruning of large vision language models: A comprehensive study on pruning dynamics, recovery, and data efficiency. *International Journal of Computer Vision (IJCV)*, 2026. arXiv:2604.24380.

Patrick Amadeus Irawan, Erland Hilman Fuadi, Shanu Kumar, Alham Fikri Aji, and Yova Kementchedjhieva. Lingudistill: Recovering linguistic ability in vision-language models via selective cross-modal distillation. *arXiv preprint arXiv:2604.00829*, 2026.

Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. Compact language models via pruning and knowledge distillation. *arXiv preprint arXiv:2407.14679*, 2024.